

Why Pipeline Accuracy Compounds ($0.9 \times 0.9 = 0.81$), by Hand

Every stage must succeed, so the system multiplies — why a chain of “good” agents is a “mediocre” system, counted stage by stage.

What this document does. It takes the one tiny multi-stage system from Module 3.2 — a query passing through a *router* and then a *specialist* — and asks the only question that matters end-to-end: *what fraction of queries does the whole pipeline get right?* We fix each stage’s accuracy, derive why a multi-stage agent’s success probability is the *product* of its stages, and then watch that product fall: router $0.90 \times$ specialist $0.90 = 0.81$. Nothing is hand-waved. Each number is reproduced by the small Python program in Appendix A. We then tabulate system accuracy against chain length, and invert the formula to find the per-stage reliability a target demands (for 0.95 over 10 stages you need 0.99488 per stage).

Where this sits. This is **Topic 2 of Module 3.2**. It is the end-to-end (surface ⑥, “whole-system”) counterpart to Topic 1, which scored the router alone: there the router read 0.856 accuracy; here that very number becomes the *first factor* in the product. Its companions: Topic 1 (router accuracy, the confusion matrix) and Module 0.0 (the ReAct cost loop, whose step count is the same k that compounds here).

Concepts covered, in order: the all-stages-must-succeed model $\cdot$ independence and the product rule $\cdot$ system success $= \prod_i a_i \cdot 0.9 \times 0.9 = 0.81 \cdot$ the uniform case $a^k \cdot$ system accuracy vs. chain length $k \cdot$ inverting to the required per-stage $a = T^{1/k} \cdot$ why long chains demand near-perfect components.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: a pipeline succeeds only if every stage does

A multi-agent system is a chain. A query enters the router, the router hands to a specialist, the specialist may hand to a synthesizer, and so on. The user only sees success if *the whole chain got it right*: a perfect specialist behind a router that misroutes still produces a wrong answer. So the system’s success is a logical AND across stages — and AND, on probabilities, multiplies.

The quantities. We give each stage a single number: its probability of doing its own job correctly, in isolation. These are illustrative round numbers, chosen so the products are followable on paper; the mechanics are identical for any values.

Quantity	Symbol	Value here
Number of stages in the chain	k	2 (router $\rightarrow$ specialist), then 5, 10, 20
Accuracy of stage i , in isolation	a_i	e.g. 0.90, 0.95, 0.99
Per-stage accuracy (uniform case)	a	all a_i equal
System (end-to-end) accuracy	S	$\prod_i a_i$
System target to hit	T	e.g. 0.95

Notation. $a_i \in [0, 1]$ is the chance stage i succeeds *given its input was correct*. S is the chance

all stages succeed in sequence. We assume the stages are *independent*: one stage’s success does not change another’s per-stage rate. That assumption is discussed honestly at the end; it makes the arithmetic exact and the bound optimistic.

Intuition

Each stage is a filter that lets through only the queries it handled correctly. Pour 100 queries into a 0.90 router and 90 come out the right side; pour those 90 into a 0.90 specialist and $0.90 \times 90 = 81$ survive. Success is what makes it through *every* filter, so the fractions multiply. Two “A-minus” stages do not make an A-minus system — they make a B-minus one.

2 Step 1 — independence and the product rule

If stage 1 succeeds with probability a_1 and stage 2 succeeds with probability a_2 *independently*, the probability that *both* succeed is, by the multiplication rule for independent events,

$$\Pr[\text{stage 1 and stage 2 both succeed}] = a_1 a_2. \quad (1)$$

Extend stage by stage. The system succeeds only if stage 1 *and* stage 2 *and* ... *and* stage k all succeed; independence lets us multiply all k factors:

$$S = \prod_{i=1}^k a_i \quad \text{and in the uniform case } a_i = a : \quad S = a^k. \quad (2)$$

Mini-example — this operation in isolation

The two-stage support pipeline. Router accuracy $a_1 = 0.90$, specialist accuracy $a_2 = 0.90$. End to end: $S = 0.90 \times 0.90 = 0.81$. ✓ Read it as a count: of 100 queries, the router correctly routes 90; of those 90, the specialist correctly resolves 90%, i.e. 81. The system is right on 81 — so it is **9 points worse than either component**, even though nothing in either component changed. The loss, $0.90 - 0.81 = 0.09$, is exactly the queries the router got right but the specialist then dropped (and vice versa is already excluded).

3 Step 2 — the same loss, stacked: a^k down the chain

The product does not just dip once — it compounds with every stage you add. Take a uniform chain where each stage is a strong 0.95. Multiply it out five times:

$$0.95^5 = 0.95 \cdot 0.95 \cdot 0.95 \cdot 0.95 \cdot 0.95 = 0.7738.$$

Five “95%” stages make a 77.4% system. Make each stage near-perfect at 0.99 and the same five-stage chain reaches $0.99^5 = 0.9510$ — only then is the *system* as good as a single 0.95 stage was. Tabulating the uniform case $S = a^k$ across chain length and per-stage accuracy:

chain length k	$a = 0.90$	$a = 0.95$	$a = 0.99$
1	0.9000	0.9500	0.9900
2	0.8100	0.9025	0.9801
3	0.7290	0.8574	0.9703
5	0.5905	0.7738	0.9510
10	0.3487	0.5987	0.9044
20	0.1216	0.3585	0.8179

Sample check, $a = 0.90$, $k = 5$: $0.9^2 = 0.81$, $0.9^4 = 0.81^2 = 0.6561$, $\times 0.9 = 0.59049$. ✓ Read the columns top to bottom and the collapse is plain: at 0.90 per stage a 10-stage agent is right barely a third of the time (0.3487); even at a heroic 0.99 per stage, a 20-stage chain has bled down to 0.818.

Watch out

The drop is *not* linear, so intuition built on adding stages misleads. Going from $k = 1$ to $k = 2$ at $a = 0.90$ costs 9 points ($0.90 \rightarrow 0.81$); going from $k = 10$ to $k = 20$ costs another $0.349 - 0.122 = 0.227$ — because you are multiplying an already-shrunk number by 0.9 ten more times. Each new stage multiplies what is left, so the absolute losses look erratic while the *ratio* stays fixed at a per stage. “Just add one more agent” is never free; it scales the whole system accuracy by that agent’s a .

4 Step 3 — inverting it: the per-stage accuracy a target demands

Turn the question around. You have a system *target* T and a chain of k stages; what per-stage accuracy a does each stage need? Set $a^k = T$ and take the k -th root:

$$\boxed{a = T^{1/k}} \quad (3)$$

This is the design equation for reliability. Work the headline case: a 0.95 system over 10 independent stages needs each stage at

$$a = 0.95^{1/10} = 0.99488.$$

✓ Verify forward: $0.99488^{10} = 0.94996 \approx 0.95$. So to ship a system that is right 95% of the time across ten hops, **every single stage must clear 99.5%** — a bar most individual LLM components do not meet. The required per-stage accuracy for several targets and lengths:

system target T	stages k	needed per-stage $a = T^{1/k}$	check a^k
0.95	5	0.98979	0.95000
0.95	10	0.99488	0.95000
0.95	20	0.99744	0.95000
0.90	10	0.98952	0.90000
0.99	5	0.99799	0.99000

Intuition

The root flips the compounding into a demand. Where a^k *punishes* you for length, $T^{1/k}$ *prices* it: the longer the chain, the closer to 1 each stage must crawl. Doubling the chain from 10 to 20 stages at a 0.95 target lifts the required per-stage accuracy from 0.99488 to 0.99744 — you must roughly *halve each stage’s error rate* just to hold the same system target while adding stages. There are only two ways to win: make each stage better, or use fewer stages. The formula offers no third option.

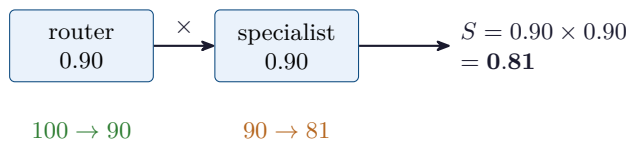
5 Why long chains demand near-perfect components

Two levers, and only two, set system accuracy in Equation (2):

- **Per-stage accuracy a .** Because $S = a^k$, raising a helps, but its effect is *exponentiated* by k — which is why the jump from 0.95 to 0.99 stages rescues the 5-stage chain from 0.774 to 0.951.
- **Chain length k .** Every stage you add multiplies S by another $a < 1$, so S falls geometrically in k . Fewer stages is often the cheapest reliability win available.

This is the structural argument for keeping agent graphs short and each node strong. A sprawling 20-agent pipeline of merely “good” (0.95) components is a 0.36 system — it fails the majority of the time, not because any agent is broken, but because $0.95^{20} = 0.3585$. The fix is never “add a checker agent” for free; the checker is one more factor in the product unless it genuinely raises the effective a of the stage it guards.

6 The chain, drawn



Each box passes through only the fraction it handles correctly; the success probabilities multiply along the arrow. Of 100 queries, 90 survive the router and 81 survive the specialist behind it — the system’s 0.81 is the area of the chain, not the height of any one box.

7 Compounding cheat-sheet

System success (independent stages)	$S = \prod_{i=1}^k a_i$
Uniform chain	$S = a^k$
Two-stage headline	$0.90 \times 0.90 = 0.81$
Five strong stages	$0.95^5 = 0.7738$
Five near-perfect stages	$0.99^5 = 0.9510$
Required per-stage for target T	$a = T^{1/k}$
0.95 over 10 stages	$a = 0.95^{1/10} = 0.99488$
Two levers, only two	raise a , or shrink k

8 An honest word about these numbers

The per-stage accuracies here (0.90, 0.95, 0.99) are illustrative round numbers, picked so the products are clean by hand. Two caveats keep this honest. First, real stages are not perfectly *independent*: a hard query tends to be hard for several stages at once (positive correlation), which makes the true system accuracy *differ* from the bare product — sometimes a little better, often worse once a misroute poisons everything downstream. Second, “success” is rarely all-or-nothing; partial credit and recovery (a specialist that handles a slightly-wrong handoff) soften the fall. What is *not* illustrative is the structure: when every stage must succeed and stages are roughly independent, system accuracy is the *product* of stage accuracies, so it decays geometrically in chain length and demands $a = T^{1/k}$ per stage to hit a target. Change the numbers and the table moves; the shape — multiply, then watch it fall — does not. That is why error compounding forces either fewer stages or much higher per-stage reliability.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the two-stage headline, the a^k scaling table, and the required-per-stage inversion; change `stages`, `ks`, or the targets to explore your own pipeline.

```

1  # compounding.py -- reproduces every number in "Why Pipeline Accuracy Compounds".
2  from math import prod
3
4  def system_acc(stage_accs): # S = product of stage accuracies
5      return prod(stage_accs)
6
7  def needed_per_stage(target, k): # invert a^k = T -> a = T**(1/k)
8      return target ** (1.0 / k)
9
10 # --- two-stage headline: router x specialist ---
11 print(f"0.90 x 0.90 = {system_acc([0.90, 0.90]):.4f}")
12
13 # --- uniform chain S = a**k across length and per-stage accuracy ---
14 print("k a=0.90 a=0.95 a=0.99")
15 for k in (1, 2, 3, 5, 10, 20):
16     print(f"{k:<4d} {0.90**k:.4f} {0.95**k:.4f} {0.99**k:.4f}")
17
18 # --- required per-stage accuracy to hit a system target ---

```

```

19 for T, k in [(0.95, 5), (0.95, 10), (0.95, 20), (0.90, 10), (0.99, 5)]:
20     a = needed_per_stage(T, k)
21     print(f"T={T} over k={k}: need a={a:.5f} (check a**k={a**k:.5f})")
22
23 # Expected output:
24 # 0.90 x 0.90 = 0.8100
25 # k a=0.90 a=0.95 a=0.99
26 # 1 0.9000 0.9500 0.9900
27 # 2 0.8100 0.9025 0.9801
28 # 3 0.7290 0.8574 0.9703
29 # 5 0.5905 0.7738 0.9510
30 # 10 0.3487 0.5987 0.9044
31 # 20 0.1216 0.3585 0.8179
32 # T=0.95 over k=5: need a=0.98979 (check a**k=0.95000)
33 # T=0.95 over k=10: need a=0.99488 (check a**k=0.95000)
34 # T=0.95 over k=20: need a=0.99744 (check a**k=0.95000)
35 # T=0.9 over k=10: need a=0.98952 (check a**k=0.90000)
36 # T=0.99 over k=5: need a=0.99799 (check a**k=0.99000)

```

— end of document —